

HPC Unit 1

HPC Unit 1 – Fundamentals of Quantitative Design and Analysis

Classes of Computers

There are five mainstream computing classes:

1. Personal Mobile Device (PMD)

Wireless devices with multimedia interfaces — smartphones, tablets, etc.

- **Key concern:** Minimize memory (memory = significant portion of cost)
- **Applications:** Web-based, media-oriented

2. Desktop Computing

Used by an individual; best price-to-performance ratio.

- **Key concern:** Price-performance
- Highest-performance and cost-reduced microprocessors appear here first

3. Servers

Multi-user systems accessed over a network (e.g., ATM servers, airline reservation).

- **Key concerns:**
 - **Availability** – must run 24/7; failure is catastrophic
 - **Scalability** – must grow with demand (compute, memory, storage, I/O)
 - **Throughput** – transactions per second, web pages per second

4. Clusters / Warehouse-Scale Computers (WSC)

Collections of desktops/servers on a LAN acting as one large computer. Each node runs its own OS; communication via networking protocols.

- **Key concerns:** Price-performance, power, availability
- Supercomputers are related but differ in workload type

5. Embedded Computers

A computer inside a device running a *fixed*, predetermined application (microwave, washing machine, networking switches, cars).

- **Key concern:** Minimum price at required performance
- Widest range of processing power and cost (8-bit to high-performance)
- Unlike PMDs, embedded computers **cannot** run externally developed software

Classes of Parallelism and Parallel Architectures

Application-Level Parallelism

Type	Definition	Example
Data-Level Parallelism (DLP)	Many data items operated on simultaneously	Applying the same function across an array of numbers
Task-Level Parallelism (TLP)	Independent tasks that can run largely in parallel	A web server handling multiple client requests simultaneously

Hardware Exploitation of Parallelism

1. **Instruction-Level Parallelism (ILP)** — Exploits DLP at modest levels using *pipelining* and *speculative execution*
2. **Vector Architectures / GPUs** — Exploit DLP by applying *one instruction* to a *collection of data* in parallel (SIMD)
3. **Thread-Level Parallelism** — Exploits DLP or TLP in tightly coupled hardware; parallel threads interact
4. **Request-Level Parallelism** — Exploits parallelism among decoupled tasks (e.g., data centers)

Flynn's Classification

Michael Flynn classified computer architectures based on instruction and data streams:

Class	Full Name	Description	Example
SISD	Single Instruction, Single Data	Standard sequential computer; ILP can be exploited	Uniprocessor
SIMD	Single Instruction, Multiple Data	Same instruction on multiple data streams; exploits DLP	GPUs, vector processors, multimedia extensions

Class	Full Name	Description	Example
MISD	Multiple Instruction, Single Data	No practical commercial implementation	—
MIMD	Multiple Instruction, Multiple Data	Each processor fetches its own instructions and data; exploits TLP	Multi-core CPUs, clusters

Note: Tightly coupled MIMD exploits **thread-level** parallelism; loosely coupled MIMD exploits **request-level** parallelism. VLIW is an example of loosely coupled MIMD.

Defining Computer Architecture

Computer architecture has three layers:

1. Instruction Set Architecture (ISA)

The boundary between hardware and software — defines the instructions a processor can execute.

2. Organization (Microarchitecture)

High-level design decisions: memory system, interconnects, CPU internals (ALU, branching, data transfer).

Example: AMD Opteron and Intel Core i7 both implement x86 ISA, but have very different pipeline and cache organizations.

3. Hardware (Implementation)

Detailed hardware realization — circuit design, packaging, clock rates.

Example: Intel Core i7 vs. Intel Xeon 7560 — nearly identical microarchitecture but differ in clock rate and memory system.

Design Requirements

When designing a new computer, the following must be considered:

1. **Functional Requirements** — Target application domain (PMD vs. Server vs. Embedded, etc.)
2. **Software Compatibility** — Existing software ecosystem for an ISA; new computers may implement existing ISAs to leverage available software

3. **OS Requirements** — Support for memory management: *pages* (fixed size) or *segments* (variable size)
 4. **Standards** — Marketplace standards like PCI, SCSI, Serial ATA
-

Instruction Set Architecture (ISA)

1. Class of ISA

- **Register-Memory ISA:** Memory can be accessed as part of many instructions → Example: x86 (80x86)
- **Load-Store ISA:** Memory is accessed *only* via load/store instructions; ALU has no direct memory access → Example: MIPS, RISC-V, ARM

2. Memory Addressing

- **Byte addressable:** 80x86, ARM, MIPS — most common
- **Alignment:** An access to an object of s bytes at address A is aligned if:

$$A \bmod s = 0$$

ARM and MIPS require alignment; 80x86 doesn't require it but is faster when aligned.

3. Addressing Modes

How operand addresses are specified:

Mode	Description	Example
Register	Operand is in a register	R1
Immediate	Constant value	#5
Displacement	Base register + constant offset	100(R1) → address = R1 + 100
Absolute (80x86)	No register, fixed address	MOV AX, [1000]
Based Indexed + Displacement (80x86)	Two registers + offset	[R1 + R2 + 4]
Scaled Index + Displacement (80x86)	Register + (register × size) + offset	[R1 + R2*4 + 8]

4. Operand Sizes

Size	Type
8-bit	ASCII character
16-bit	Unicode character, half-word
32-bit	Integer, word
64-bit	Double word, long integer
32-bit float	IEEE 754 single precision
64-bit float	IEEE 754 double precision
80-bit float	Extended double precision (80x86 only)

5. Operation Categories

- Data transfer
- Arithmetic and logical
- Control flow
- Floating point

6. Control Flow Instructions

All ISAs support: conditional branches, unconditional jumps, procedure calls and returns.

All use **PC-relative addressing**: branch address = PC + offset field.

Key differences:

- MIPS/RISC-V: branches test *register contents* (e.g., `BEQ`, `BNE`)
- 80x86/ARM: branches test *condition code bits* set by arithmetic ops (e.g., `JC`, `JNC`)
- ARM/MIPS: return address stored in a *register*
- 80x86: return address pushed onto a *memory stack*

7. Instruction Encoding

Type	Description	Advantage
Fixed Length	All instructions are 32 bits (ARM, MIPS)	Simple decoding; PC update is trivial (PC + 4)
Variable Length	1–18 bytes (80x86), variable (RISC-V)	Smaller code size; more compact programs

Dependability

Core Concept

Dependability is the quality of delivered service that justifies relying on a system.

System is dependable if: Delivered Service $\approx$ Specified Service

Fault → Error → Failure Chain

Term	Definition	Example
Fault	A defect in the system	Bug in <code>add(5, 3)</code> function
Error	Incorrect internal state caused by fault	<code>add(5, 3)</code> returns wrong value
Failure	Delivered service deviates from specification	Calling code gets wrong result and crashes

Key insight: A fault doesn't always lead to a failure. Example: if the result of `add(5, 3)` is never used in a critical path, the error may never propagate to a failure.

Service States

Systems alternate between:

1. **Service Accomplishment** — service delivered as specified
2. **Service Interruption** — delivered service differs from SLA

Transitions:

- **Failure:** accomplishment → interruption
- **Restoration:** interruption → accomplishment

Service Level Agreements (SLA) / Service Level Objectives (SLO): Contracts that define acceptable service; used to decide if a system is "up" or "down."

Dependability Metrics

Mean Time To Failure (MTTF)

Average time a system operates continuously before the next failure.

$MTTF = \text{Average time of continuous service accomplishment}$

Example: A system that runs for 1 year then fails → $MTTF = 1 \text{ year}$.

Rate of Failures (FIT — Failures in Time)

$$FIT = \frac{10^9}{MTTF \text{ (in hours)}}$$

Example: MTTF = 1,000,000 hours $\rightarrow$ FIT = $10^9/10^6 = 1000$ FIT

Mean Time To Repair (MTTR)

Average time taken to restore service after a failure.

Mean Time Between Failures (MTBF)

$$\text{MTBF} = \text{MTTF} + \text{MTTR}$$

Module Availability

Fraction of total time the system is in the accomplishment state.

$$\text{Availability} = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}$$

Quantitative Principles of Computer Design

1. Take Advantage of Parallelism

Level	Example
Bit-level	4-bit $\rightarrow$ 8-bit $\rightarrow$ 16-bit $\rightarrow$ 32-bit ALUs
Instruction-level (ILP)	Pipelining — overlap instruction execution
Data-level	Spread data across disks for parallel reads/writes
Thread-level	Multi-core processors
Logic-level	Carry Look-Ahead Adder (CLA) — uses parallelism instead of ripple carry

2. Principle of Locality

Programs tend to reuse recently accessed data and instructions. About **90% of execution time** is spent accessing memory.

Type	Definition	Example
Temporal Locality	Recently accessed items are likely to be accessed again soon	A variable <code>sum</code> updated in every loop iteration
Spatial Locality	Items near recently accessed locations are likely to be accessed soon	Accessing <code>array[0]</code> , <code>array[1]</code> , <code>array[2]</code> ... sequentially

This principle motivates **memory hierarchies** — small, fast memory close to CPU; large, slow memory far away.

3. Focus on the Common Case

"Making the common case fast will tend to enhance performance more than optimizing the rare case."

Example: Adding two numbers in a processor — overflow is rare. Optimize the no-overflow path, handle overflow as a special case.

Amdahl's Law formalizes this:

$$\text{Speedup}_{\text{overall}} = \frac{\text{Exec Time}_{\text{old}}}{\text{Exec Time}_{\text{new}}} = \frac{1}{(1 - f) + \frac{f}{s}}$$

Where:

- f = fraction of time the enhanced feature is used
- s = speedup of the enhanced feature

Takeaway: If only 10% of execution uses a feature, even making that feature infinitely fast yields at most a 1.11× overall speedup.

Cache Basics

What is Cache?

Cache is a small, fast memory between the processor and main memory that makes main memory *appear faster* than it actually is.

- **Primary (L1) Cache:** Inside the processor chip
- **Secondary (L2) Cache:** External to processor (or on-chip but separate)

Memory Hierarchy Terminology

Term	Definition
Block	Minimum unit of data transferred between hierarchy levels
Hit	Requested data is found in the upper (closer to CPU) level
Hit Rate	Fraction of accesses found at the upper level
Hit Time	Time to check if access is a hit + time to deliver data to CPU
Miss	Requested data is NOT in the upper level
Miss Rate	1 – Hit Rate

Term	Definition
Miss Penalty	Time to fetch the block from the lower level + deliver it to CPU

Cache Operation (Basic)

1. CPU requests a word
2. Check cache:
 - **Hit** → deliver data directly from cache
 - **Miss** → fetch block from main memory into cache, then deliver to CPU

Direct Mapped Cache Concept

Each main memory block maps to **exactly one** cache location.

$$\text{Cache Block Index} = \text{Memory Block Address} \bmod \text{Number of Cache Blocks}$$

Example: Cache with 8 blocks; main memory has 32 blocks:

- Blocks 0, 8, 16, 24 → all map to cache block 0
- Blocks 1, 9, 17, 25 → all map to cache block 1

If cache size = 2^n blocks, then the cache index = **lower n bits** of the memory address.

Address Breakdown

For a processor with m -bit addresses:

$$\text{Address} = \underbrace{\text{TAG}}_{\text{upper bits}} \mid \underbrace{\text{Index}}_{n \text{ bits}} \mid \underbrace{\text{Block Offset}}_{k \text{ bits}}$$

Where:

- $n = \log_2 \left(\frac{\text{Cache size in blocks}}{\text{Associativity}} \right)$ (Associativity = 1 for Direct Mapping)
- $k = \log_2(\text{Block size in bytes})$
- TAG bits = $m - (n + k)$

Valid Bit

Each cache entry has a **Valid bit**:

- **V = 1:** Entry contains valid data
- **V = 0:** Entry is empty or stale (e.g., at startup)

Cache Entry Size (Write-Through)

$$\text{Size per entry (bits)} = \text{Block size (bits)} + \text{TAG bits} + 1 \text{ (valid bit)}$$

For **write-back**, add 1 dirty bit:

$$\text{Size per entry (bits)} = \text{Block size (bits)} + \text{TAG bits} + 1 \text{ (valid)} + 1 \text{ (dirty)}$$

Example: Calculating Cache Parameters

Problem: Direct-mapped cache with 1K blocks, each block = 1 word (4 bytes). 32-bit addressing.

a) Index and block offset bits:

$$n = \log_2(1024) = 10 \text{ bits (Index)}$$

$$k = \log_2(4) = 2 \text{ bits (Block offset = Byte offset)}$$

b) TAG bits:

$$\text{TAG} = 32 - (10 + 2) = 20 \text{ bits}$$

c) Size per cache entry (write-through):

$$= 32 + 20 + 1 = 53 \text{ bits}$$

d) Total cache size:

$$= 1024 \times 53 = 54272 \text{ bits} \approx 53 \text{ Kbits}$$

e) For address 0x2E101A20:

$$= 0010 \ 1110 \ 0001 \ 0000 \ 0001 \ | \ \underbrace{1010 \ 0010 \ 00}_{10 \text{ bits}} \ 00$$

$$\text{Index} = 0x288 = 648\text{th entry}$$

Handling a Cache Miss

Steps when an instruction cache miss occurs:

1. PC has already been incremented, so the faulting instruction address = **PC - 4**
 2. Stall the processor; send address to main memory
 3. Write fetched data into cache: update data, write TAG from upper address bits, set Valid bit = 1
 4. Restart instruction execution (now it will be a hit)
-

Handling Writes

The Problem: Inconsistency

If a store instruction writes *only* to cache (not main memory), the two copies diverge — **inconsistency**.

Write-Through

Always write to both cache and main memory simultaneously.

- **Merits:** Simple; cache always consistent with memory; no dirty bit needed; read misses never cause writes to lower level
- **Demerits:** Every write goes to slow main memory (~100 cycles penalty), causing processor stalls

Write Buffer (Write-Through Optimization)

A queue between cache and main memory:

1. Processor writes to cache AND write buffer
2. Processor continues immediately (doesn't wait)
3. Memory controller drains write buffer to main memory in background

Problem: If write buffer fills faster than memory can drain it, the processor must stall.

Write-Back

Write only to cache. Write to main memory only when the block is evicted (replaced).

A **Dirty bit (D)** is added to each cache entry:

- **D = 1:** Block has been modified; main memory is out of date
- **D = 0:** Block is clean; cache and memory are consistent

On a replacement:

1. If $D = 1$ → write dirty block back to main memory *before* loading new block
2. If $D = 0$ → simply overwrite the cache entry

Advantages over Write-Through:

- Writes happen at cache speed
- Multiple writes to the same block → only **one** write to main memory
- Less memory bandwidth → useful in multiprocessors and embedded systems

Disadvantage:

- Even read misses can trigger a write to lower level (if evicted block is dirty)

- More complex to implement

Write Miss Policies

Policy	Behavior on Write Miss	Typical Use
Write Allocate	Load block into cache, then write (like a read miss)	Used with write-back
No-Write Allocate	Write directly to main memory; cache is NOT updated	Used with write-through

Write Allocate Example (sequence: Write[100], Write[100], Read[200], Write[200], Write[100]):

Operation	Result	Explanation
Write [100]	Miss	Load block from memory, then write to cache
Write [100]	Hit	Block now in cache
Read [200]	Miss	Load block from memory
Write [200]	Hit	Block in cache
Write [100]	Hit	Block still in cache

Hit Rate = $3/5 = 0.6$

No-Write Allocate Example (same sequence):

Operation	Result	Explanation
Write [100]	Miss	Write directly to main memory, no cache update
Write [100]	Miss	Write directly to main memory again
Read [200]	Miss	Load into cache
Write [200]	Hit	Block in cache
Write [100]	Miss	Cache never got block [100]

Hit Rate = $1/5 = 0.2$

Set Associative and Fully Associative Cache

Motivation: Problem with Direct Mapping

In direct mapping, if two frequently used blocks map to the **same** cache index, they keep evicting each other — **thrashing** — even if other cache lines are free.

Set Associative Cache

Cache is divided into **sets**, each set holds n blocks (n-way set associative).

$$\text{Set Index} = \text{Block Address} \mod \text{Number of Sets}$$

A block can go into *any* of the n ways within its assigned set.

$$\text{Number of Sets} = \frac{\text{Cache size in blocks}}{\text{Associativity}}$$

Address breakdown for n-way set associative cache:

$$n_{\text{index}} = \log_2(\text{Number of Sets}), \quad k = \log_2(\text{Block size in bytes})$$

$$\text{TAG} = m - (n_{\text{index}} + k)$$

Note: As associativity increases, the number of sets decreases, index bits decrease, and TAG bits increase.

Fully Associative Cache

A block can go **anywhere** in the cache. Equivalent to 1 set with M blocks (M-way set associative).

- **Lowest conflict miss rate**
- **Highest hit time** — every TAG must be checked in parallel (requires M comparators)

Comparison: Miss Rate vs. Associativity

Organization	Conflict Miss Rate	Hit Time
Direct Mapped	Highest	Lowest
Set Associative (n-way)	Moderate	Moderate
Fully Associative	Lowest	Highest

Example: 4-block cache, sequence 0, 8, 0, 6, 8:

- Direct mapped: **5 misses**
- 2-way set associative (LRU): **4 misses**
- Fully associative (LRU): **3 misses**

Block Replacement Policies

Policy	Description
LRU (Least Recently Used)	Evict the block unused for the longest time; tracked with LRU bit per set
Random	Evict a randomly chosen block
FIFO	Evict the oldest loaded block

LRU Bit (2-way set associative):

- **LRU bit = 1:** Way 1 was least recently used → evict Way 1 on miss
- **LRU bit = 0:** Way 0 was least recently used → evict Way 0 on miss

Size of Tags vs. Associativity (Example)

Given: 4096-block cache, 4-word block, 32-bit addressing

Organization	Index bits	TAG bits	Total TAG bits
Direct Mapped	12	16	$16 \times 4096 = \mathbf{66 \text{ Kbits}}$
2-way	11	17	$17 \times 2 \times 2048 = \mathbf{70 \text{ Kbits}}$
4-way	10	18	$18 \times 4 \times 1024 = \mathbf{74 \text{ Kbits}}$
Fully Associative	0	28	$28 \times 4096 = \mathbf{115 \text{ Kbits}}$

Key Insight: Each doubling of associativity reduces index bits by 1, increases TAG bits by 1, and requires more comparators.

Cache Miss Classification (3 Cs)

1. Compulsory (Cold Start) Misses

The **very first** access to any block — it was never in cache before.

- Independent of cache size and associativity
- **Reduced by:** Increasing block size (exploits spatial locality — when a miss occurs, neighboring words are also fetched)

2. Capacity Misses

Cache is full; a block was evicted due to **lack of space**, and then referenced again.

- Occurs even in fully associative caches (so not caused by mapping)
- **Reduced by:** Increasing cache size

3. Conflict Misses

A block was evicted because its assigned **set was full**, even though other sets had free space — due to mapping restrictions.

- Only occurs in direct-mapped and set-associative (not fully associative)
- **Reduced by:** Increasing associativity

Summary Table

Optimization	Compulsory	Capacity	Conflict	Overall
Larger Block Size	Decreases	May Increase	May Increase	Varies
Larger Cache	No Effect	Decreases	May Decrease	Decreases
Higher Associativity	No Effect	No Effect	Decreases	Decreases

Which cache mapping is most commonly used? Set-associative is the most popular: it reduces conflict misses compared to direct mapping, and reduces search time (hit time) compared to fully associative.

Cache Performance

CPU Execution Time with Cache

$$\text{CPU Time} = (\text{CPU cycles} + \text{Memory stall cycles}) \times \text{Clock cycle time}$$

$$\text{Memory stall cycles} = \text{Read stalls} + \text{Write stalls}$$

$$\text{Read stall cycles} = \frac{\text{Read accesses}}{\text{Program}} \times \text{Read miss rate} \times \text{Miss penalty}$$

For write-through with write buffer:

$$\text{Write stall cycles} = \frac{\text{Writes}}{\text{Program}} \times \text{Write miss rate} \times \text{Miss penalty} + \text{Write buffer stalls}$$

Simplified (assuming same penalty for reads and writes):

$$\text{Memory stall cycles} = \frac{\text{Memory accesses}}{\text{Program}} \times \text{Miss rate} \times \text{Miss penalty}$$

Or equivalently:

$$\text{Memory stall cycles per instruction} = \text{Misses per instruction} \times \text{Miss penalty}$$

Example

Given:

- CPI = 1.0 (assuming perfect cache)
- Load/Store = 50% of instructions
- Miss penalty = 25 clock cycles
- Miss rate = 2%

Case 1: Perfect cache

$$\text{CPI}_{\text{ideal}} = 1.0$$

Case 2: Real cache

$$\text{Memory stall cycles/instruction} = 1.0 \times 2$$

One instruction fetch + 0.5 data accesses per instruction on average (since only 50% are loads/stores)

$$\text{Actual CPI} = 1.0 + 0.75 = 1.75$$

Processor speedup degraded due to cache misses:

$$\text{Speedup}_{\text{ideal vs real}} = \frac{1.75}{1.0} = 1.75 \times \text{slower}$$

Key insight: If you speed up the CPU (lower CPI) without improving memory, memory stalls become a *larger* fraction of total time.

Average Memory Access Time (AMAT)

$$\text{AMAT} = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$$

Example:

- Single off-chip cache: Hit Time = 2 ns, Hit Rate = 98%, Main Memory = 40 ns

$$\text{AMAT} = 2 + 0.02 \times 40 = 2.8 \text{ ns}$$

Split Cache

Separate instruction cache and data cache operating in parallel.

Advantages:

1. Instruction fetch and data access can occur in the **same cycle** (critical for pipelining)
2. Double the effective bandwidth of a unified cache
3. Each cache can be optimally placed near its respective pipeline stage

Cache Optimization Techniques

$$\text{AMAT} = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$$

There are **6 basic optimizations**:

Optimization 1: Larger Block Size → Reduce Miss Rate (Compulsory)

Effect: Exploits spatial locality — neighboring words fetched together.

Miss Type	Effect
Compulsory	Decreases (more neighbors brought in per miss)
Capacity	May increase (fewer blocks fit in same-size cache)
Conflict	May increase (fewer blocks → more contention per set)

Optimal block size: 32 bytes for small caches (4 KB), 64 bytes for larger caches — these are standard sizes in modern processors.

Miss Penalty increases with larger blocks (more data to transfer on each miss).

Optimization 2: Larger Cache Size → Reduce Miss Rate (Capacity)

Miss Type	Effect
Compulsory	No effect
Capacity	Decreases
Conflict	May decrease (more blocks → fewer conflicts per set)

Tradeoff: Larger cache → higher hit time (more hardware to search).

Optimization 3: Higher Associativity → Reduce Miss Rate (Conflict)

Miss Type	Effect
Compulsory	No effect
Capacity	No effect (same total blocks)
Conflict	Decreases

Tradeoff: Higher associativity → higher hit time (MUX delay to select winning way).

Rule of thumb: 8-way set associative ≈ fully associative for most workloads.

Optimization 4: Multilevel Caches → Reduce Miss Penalty

Add L2 (and L3) caches between L1 and main memory.

$$AMAT = HT_{L1} + MR_{L1} \times MP_{L1}$$

$$MP_{L1} = HT_{L2} + MR_{L2}^{local} \times MP_{L2}$$

Local vs. Global Miss Rate:

$$MR_{L2}^{local} = \frac{\text{Misses in L2}}{\text{Accesses to L2}} = \frac{\text{Misses in L2}}{\text{Misses in L1}}$$

$$MR_{L2}^{global} = MR_{L1}^{local} \times MR_{L2}^{local} = \frac{\text{Misses in L2}}{\text{Total accesses from CPU}}$$

Global miss rate = fraction of processor memory accesses that reach main memory.

Design guidelines:

- L1 should be **small and fast** (to match CPU clock)
- L2 should be **large** (to absorb L1 misses; if $L2 \approx L1$ in size, local miss rate of L2 will be very high)
- Speed of L1 affects CPU clock rate; speed of L2 affects only the L1 miss penalty

Example:

- Base CPI = 1.0, clock = 4 GHz
- L1 miss rate = 2%, Main memory access = 100 ns = 400 cycles
- L2 hit time = 5 ns = 20 cycles, L2 reduces miss rate to main memory to 0.5%

$$\text{CPI with L1 only} = 1 + 2$$

$$\text{CPI with L2} = 1 + 2$$

$$\text{Speedup} = \frac{9}{3.4} \approx 2.6\times$$

Optimization 5: Prioritize Reads over Writes → Reduce Miss Penalty

Why reads are more critical:

- On a **read**, the processor stalls and *cannot continue* until data arrives
- On a **write**, the processor can use a write buffer and continue immediately

Read-After-Write (RAW) Hazard: Occurs when a read miss requests a block that has been recently written but the updated value is still in the write buffer (not yet in main memory or cache).

Example: Write to Mem[512], then read Mem[1024] (both map to same cache block in direct-mapped). Write buffer still holds updated Mem[512] value; cache has been

evicted. If the read miss fetches from main memory, it gets the **old** (stale) value of Mem[512].

Solutions:

1. **Wait until write buffer is empty** before servicing a read miss (simple but slow)
2. **Check write buffer on read miss:**
 - If the requested address is in the write buffer → supply data from write buffer directly ✓
 - If not → fetch from main memory normally ✓

Modern CPUs use option 2 — reads get priority over writes.

Optimization 6: Avoid Address Translation → Reduce Hit Time

Why translation is costly:

- Processor generates **virtual addresses**
- These must be translated to **physical addresses** before cache can be indexed
- Translation via TLB adds latency

Three approaches:

Scheme	Indexing	Tagging	Notes
PIPT (Physically Indexed, Physically Tagged)	Physical	Physical	Safe but slow (translation before indexing)
VIVT (Virtually Indexed, Virtually Tagged)	Virtual	Virtual	Fast but no page protection (aliasing issues)
VIPT (Virtually Indexed, Physically Tagged)	Virtual index	Physical tag	Best of both — parallel indexing + translation

VIPT Condition (for correctness):

$$\text{Page size} \geq \text{Cache size} / \text{Associativity}$$

This ensures the index bits lie within the page offset (which doesn't change during translation), so indexing can start with the virtual address while TLB translation happens in parallel.

Example: Page size = 8 KB (13-bit offset). 32 KB cache, 4-way, 64B blocks:

$$\text{Cache size} / \text{Associativity} = 32\text{KB} / 4 = 8\text{KB}$$

Since $8\text{KB} \leq 8\text{KB}$, VIPT is valid. ✓

Virtual Memory and Address Translation

Motivation

A 32-bit processor can address $2^{32} = 4$ GB of memory. Physical DRAM may be only 1 GB. **Virtual memory** bridges this gap.

- **Virtual Address Space:** All addresses a program can generate (defined by ISA)
- **Physical Address:** Actual address in DRAM
- A **translator (TLB — Translation Lookaside Buffer)** converts virtual $\rightarrow$ physical

Paging

Virtual memory is organized into fixed-size blocks called **pages** (power of 2).

$$\text{Virtual Address} = \underbrace{\text{Virtual Page Number (VPN)}}_{\text{upper bits}} \mid \underbrace{\text{Page Offset}}_{\text{lower bits}}$$

$$\text{Page offset bits} = \log_2(\text{Page size in bytes})$$

Example: 16-bit virtual address, page size = 2 KiB:

$$\text{Page offset bits} = \log_2(2048) = 11 \text{ bits}$$

$$\text{VPN bits} = 16 - 11 = 5 \text{ bits}$$

$$\text{Number of pages} = 2^5 = 32 \text{ pages}$$

Data between **cache and main memory** is transferred in **blocks**; data between **main memory and disk** is transferred in **pages**.

Quick Reference: Formulas

Formula	Expression
Cache index bits	$n = \log_2 \left(\frac{\text{Cache blocks}}{\text{Associativity}} \right)$
Block offset bits	$k = \log_2(\text{Block size in bytes})$
TAG bits	$m - (n + k)$ where m = address bits
Bits per cache entry	Block bits + TAG bits + Valid bit (+ Dirty bit for write-back)
Total cache bits	Entries $\times$ Bits per entry
Miss rate	$1 - \text{Hit rate}$
MTBF	MTTF + MTTR
Availability	$\frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}$
AMAT (1 level)	$\text{HT} + \text{MR} \times \text{MP}$

Formula	Expression
AMAT (2 levels)	$HT_{L1} + MR_{L1} \times (HT_{L2} + MR_{L2}^{local} \times MP_{L2})$
Global MR (L2)	$MR_{L1}^{local} \times MR_{L2}^{local}$
Memory stall cycles/instruction	$\frac{\text{Memory accesses}}{\text{Instruction}} \times MR \times MP$
CPU Time	$(CPI + \text{Mem stall cycles/instr}) \times IC \times \text{Clock period}$
Amdahl's Law	$\text{Speedup} = \frac{1}{(1-f) + f/s}$
FIT	$10^9 / \text{MTTF (hours)}$